A Minimal Task-Space Arm Controller for MABEL: Pure PD, and What Gravity Compensation Buys

MABEL controller subsystem

19 July 2026

Abstract

We rebuild MABEL's arm controller from the smallest law that can hold a limb — a joint-space PD — and add task-space wrist tracking on top of it as a separate, independently measurable stage. The rewrite is motivated by a diagnosis problem rather than a performance one: in the layered controller it had become impossible to answer *which* stage was responsible when an arm misbehaved. We characterise the pure-PD baseline, show that its steady-state droop is exactly the uncompensated gravity torque (τ_g/k_p, measured against prediction per joint), and quantify what a gravity feed-forward is worth. At $k_p = 400$ the feed-forward reduces palm error from 14.2 mm to 0.38 mm — a 37× improvement — and a compensated arm at $k_p = 100$ is more accurate than an uncompensated one at $k_p = 800$, i.e. the feed-forward buys more than an eightfold stiffness increase without the stiffness.

Keywords: joint-space PD, gravity compensation, damped least squares, task-space control, redundancy resolution, MuJoCo.

1 Motivation

The previous arm controller was a layered stack: a retargeting engine, a whole-body torque engine, per-region set-points, and a facade routing between them. It worked, but it had one property that made it expensive to own — when the arm did something unexpected, the observable was a single symptom ("the arm sags", "the hand goes crazy") and there was no measurement that localised the cause. Two failures traced to that opacity: a gravity feed-forward that ran only when an operator pose frame arrived, so it never ran in the stick-only drive mode; and a set-point map that silently skipped the wrists, so the torque law re-defaulted them to the measured angle every tick — a damper, not a hold, which creeps.

This document describes the replacement baseline. The design goal is not peak performance; it is that *every stage reports its own error*.

2 Method

2.1 Joint-space PD

For each joint i in the controlled chain,

$$\tau_i \;=\; k_{p,i}\,(q_i^{*} - q_i) \;-\; k_{d,i}\,\dot{q}_i \;+\; \underbrace{g_i(q)}_{\text{optional}}\,, \tag{1}$$

clamped to the actuator's declared force range. With $g \equiv 0$ this is a pure PD. At equilibrium $\dot{q} = 0$ and the joint settles where the proportional term balances the load,

$$k_{p,i}\, e_i \;=\; \tau_{g,i} \qquad \Longrightarrow \qquad e_i \;=\; \frac{\tau_{g,i}}{k_{p,i}}, \tag{2}$$

so the steady-state droop is fully determined by the gravity torque and the gain. Equation (2) is a prediction we test directly in §3.2.

One law per joint. The MJCF drives these joints with `position` actuators, which are themselves a PD (k_p=100, k_d=20 on the arms). Attaching our controller therefore *neutralises* the servo of every joint it owns and restores it on detach. Without this, any measurement describes the sum of two controllers rather than the one under test — an easy and completely silent way to characterise the wrong system.

Gravity term — model-based, and the same on hardware. The compensation is computed from an a-priori rigid-body model and the *measured* joint angles, in numpy, with no call into the simulator:

$$\tau_{g,i}(q) \;=\; -\sum_{j \geq i} \big(\hat{z}_i \times (c_j - p_i)\big) \cdot (m_j\, \mathbf{g}), \tag{3}$$

the weight of every link distal to joint i, moment-armed about that joint's axis. The axes $\hat{z}_i$, anchors p_i and link centres of mass c_j come from forward kinematics on q; the constants m_j, the fixed link transforms and the COM offsets are baked once from the MJCF (`extract_arm_params.py`), the same class of a-priori data a URDF carries. An earlier version read MuJoCo's `qfrc_bias` — correct and one line, but privileged: it needs a live simulator, which the physical arm does not have, and it folds in contacts and external wrenches, which a feed-forward should not. Equation (3) needs only the model and the encoders, both of which exist on hardware, so *one controller runs in simulation and on the robot*. It is velocity-free by construction — no Coriolis, so no risk of cancelling the damping (1) relies on. Its agreement with MuJoCo's own gravity torque (§3.7, $< 3 \times 10^{-7}\,\mathrm{N\,m}$ over the workspace) is what licenses the swap. The finger links are lumped into their nearest arm ancestor at the hand's rest pose; with the hand fully articulated this costs at most $0.14\,\mathrm{N\,m}$ (0.9%), measured, not assumed.

Base orientation. Gravity enters through the shoulder's orientation, so a pitched torso changes the load. (3) takes the world←base rotation; on the robot it comes from the torso/lift encoders or an IMU, and it defaults to the rest-pose lean baked by the extractor rather than to a bare "upright" assumption, which would be a silent 10–20% error on the shoulder joints.

2.2 Task-space wrist tracking

The wrist target is resolved to joint angles by damped least squares. With $J \in \mathbb{R}^{3\times n}$ the palm's positional Jacobian restricted to the chain and $e = x^* - x$ the task error,

$$\Delta q \;=\; J^\top \Big(J J^\top + \lambda^2 I \Big)^{-1} e \;+\; \big(I - J^+ J\big)\, k_n\, (q_{\text{rest}} - q). \tag{4}$$

The chain is 7-DOF against a 3-DOF task, so the null space is four-dimensional and must be resolved or it drifts: two frames identical at the fingertip can differ by an elbow swung through the torso. The second term applies a weak posture pull with gain k_n.

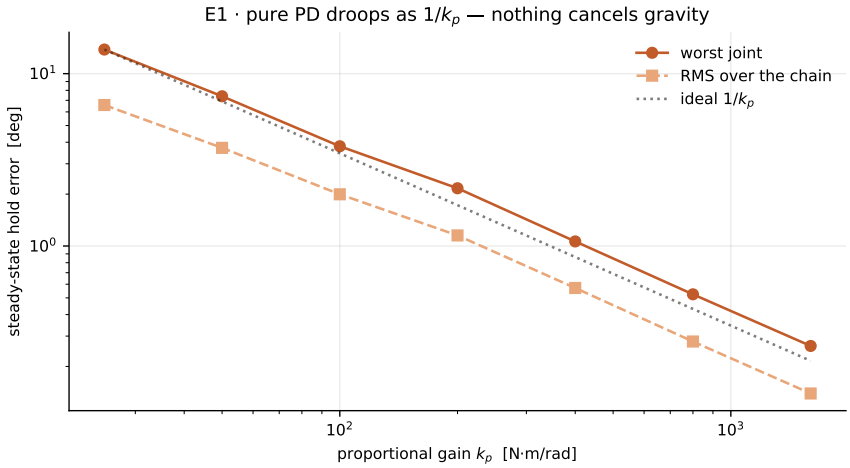
Damping is a safety parameter. Near a singularity, or for a target outside the reachable set, $JJ^\top$ is ill-conditioned and an undamped pseudo-inverse requests unbounded joint rates — the familiar failure where dragging a target past the arm's reach makes the arm explode. The $\lambda^2 I$ term bounds the step; §3.5 shows the residual growing linearly and the joints remaining inside their limits rather than diverging. λ should be tuned for stability, never for speed.

The ORCA wrist is excluded from the chain. It carries the hand and belongs to grasp orientation, not to reaching. Allowing a position-only solver to spend it is what twists the hand while the palm sits perfectly on target.

3 Experiments

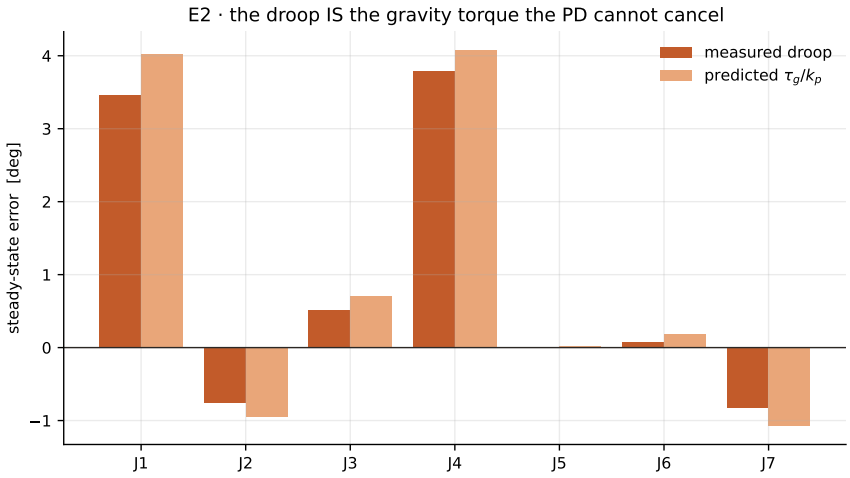
All figures are regenerated from scratch by `controller/experiments/exp_basic_controller.py` against the canonical MJCF (`simulation/mabel_mujoco/scenes/mabel_home.xml`), right arm, 2 ms timestep, 4 s of settling per steady-state measurement, $k_d = 2\sqrt{k_p}$.

3.1 Steady-state droop versus gain



The droop follows $1/k_p$ across six octaves: 13.80° at k_p=25 falling to 0.26° at k_p=1600, each doubling of gain halving the error, exactly as (2) requires.

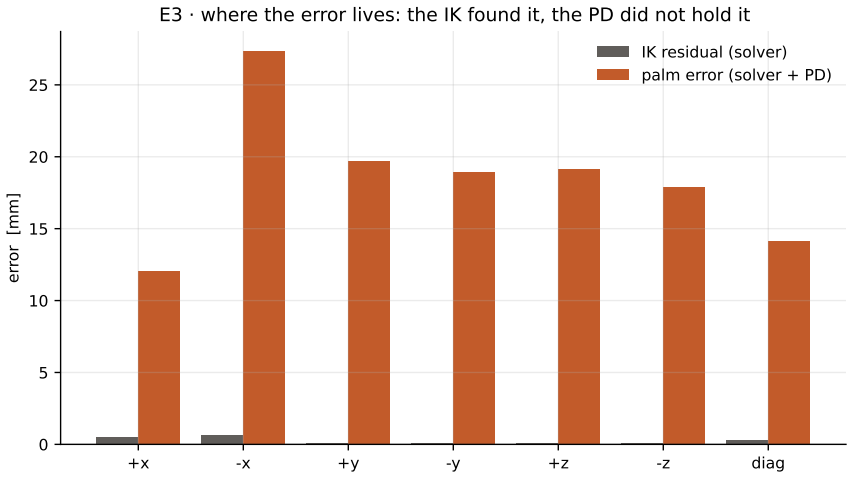
3.2 Per-joint droop against prediction



Joint	measured [°]	predicted τ_g/k_p [°]
J1	+3.460	+4.026
J2	−0.762	−0.950
J3	+0.514	+0.711
J4	+3.791	+4.073
J5	−0.000	+0.021
J6	+0.070	+0.189
J7	−0.829	−1.071

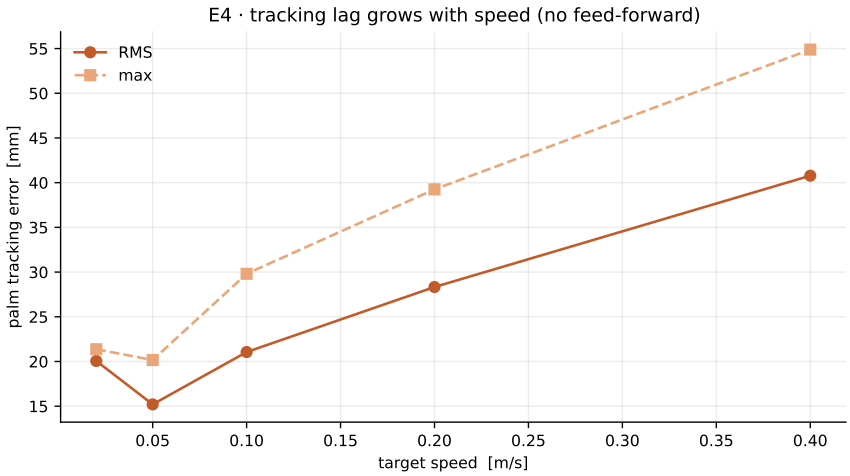
Measurement tracks prediction in sign and magnitude on every joint, and the load concentrates in J1 and J4 as expected. The systematic 10–25% shortfall is itself explained by (2): the joint droops, which changes the configuration, which reduces τ_g — the prediction is evaluated at the commanded pose, the measurement at the settled one.

3.3 Static wrist tracking, and where the error lives



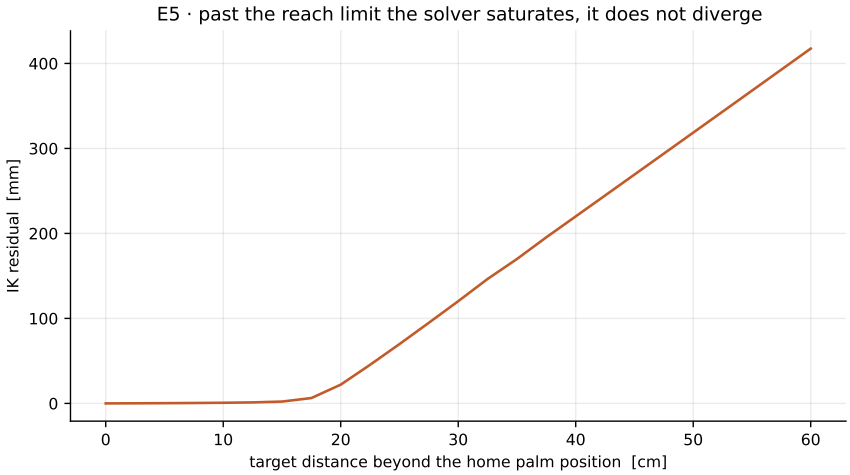
At k_p=400 over seven 10 cm offsets, the IK residual is 0.06–0.65 mm while the settled palm error is 12–27 mm. The solver is two orders of magnitude better than the arm that follows it: *the entire task-space error is the joint-space droop*. This is the diagnostic the rewrite exists to produce.

3.4 Moving targets



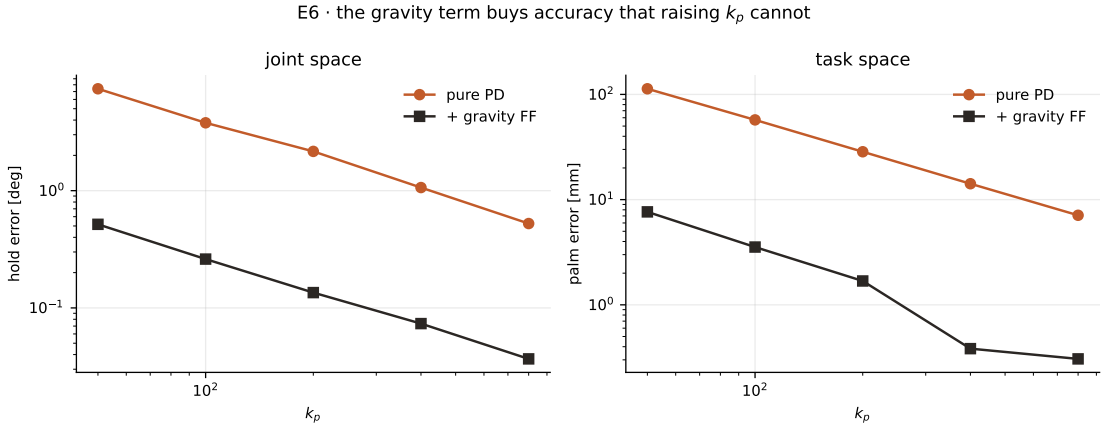
Tracking a 6 cm circle, RMS palm error grows from 20 mm at 0.02 m/s to 41 mm at 0.40 m/s. With no velocity feed-forward the lag is expected to grow with speed; the floor at low speed is again the static droop.

3.5 Behaviour past the reach limit



Pushing the target outward, the residual stays near zero to ∼15 cm beyond the home palm position and then grows *linearly* (120 mm at +0.30 m, 417 mm at +0.60 m) — the solver saturates against the joint limits and reports a large residual rather than diverging.

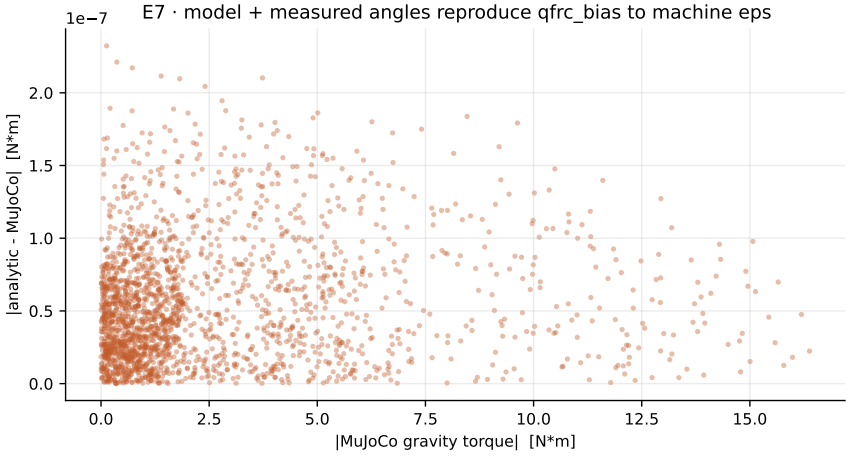
3.6 What gravity compensation buys



k_p	hold, PD [°]	hold, $+g$ [°]	palm, PD [mm]	palm, $+g$ [mm]	gain
50	7.394	0.516	113.11	7.65	14.8×
100	3.791	0.261	57.24	3.54	16.2×
200	2.162	0.135	28.51	1.69	16.9×
400	1.063	0.073	14.16	0.38	36.9×
800	0.524	0.037	7.10	0.31	23.2×

The feed-forward is worth one to two orders of magnitude at every gain. The practical statement is the cross-comparison: a compensated arm at k_p=100 (3.44 mm) outperforms an uncompensated one at k_p=800 (7.10 mm), so the gravity term buys more than an eightfold stiffness increase *and* leaves the arm compliant. Raising k_p to hide droop purchases stiffness nobody asked for and an oscillation nobody wants.

3.7 Gravity model fidelity



Over 300 random arm poses the analytic model of (3) reproduces MuJoCo's `qfrc_bias` to 2.3×10^{-7} N m against torques up to 16.4 N m — machine precision, computed at runtime in numpy from the baked constants and the joint angles alone. This is the evidence that replacing the

simulator-privileged term with a hardware-portable one costs nothing in simulation, so the same code can be trusted on the physical arm.

4 Implementation and usage

```
from mabel.basic import TaskSpaceArm
arm = TaskSpaceArm(model, data, side="right", kp=400, kd=40,
                   gravity=True).attach()          # numpy-only gravity model
arm.set_wrist_target([x, y, z])                    # world metres; None -> hold
while running:
    arm.set_base_orientation(torso_R)              # encoders/IMU on the robot
    arm.step()                                     # IK -> q*, then torque
    mujoco.mj_step(model, data)                    # (or the real arm's step)
print(arm.ik_err, arm.task_error(), arm.joint_error())
```

`mabel/basic/` is: `pd.py` (the law + actuator ownership), `wrist_ik.py` (DLS with null-space posture), `arm.py` (the composition + measurement surface), `gravity.py` (the numpy gravity model) and `arm_params.py` (constants baked by `extract_arm_params.py`). Guarded by `controller/tests/test_basic_controller.py` (15 tests): the $1/k_p$ law, the $> 5\times$ gravity improvement, torque clamping, actuator restoration, out-of-reach saturation, the velocity-independence of the gravity term, the machine-precision match to `qfrc_bias`, that `gravity.py` imports *no* simulator (AST-checked), and that the baked masses still match the MJCF.

5 Limitations

- **Position-only IK.** Palm orientation is unconstrained. Grasping needs a 6-DOF task; (4) extends to a stacked Jacobian, but the redundancy shrinks to one dimension and the posture term matters more.
- **No velocity feed-forward.** §3.4's lag is the direct consequence. A $\dot{x}^*$ term through J^+ is the obvious next step.
- **Model-based gravity, fixed payload.** $g(q)$ is only as good as the baked masses. The arm's own links are known well; a *grasped* load is not modelled, and the ORCA hand alone is 1.74 kg per wrist, so a heavy object will droop until an online payload estimate is added.
- **Single arm, no whole-body coupling.** Torso, lift and base are outside the chain, so their motion is a disturbance the PD rejects rather than a coordinated degree of freedom.
- **Simulation only.** Every number here is MuJoCo. Nothing in this document has been measured on the physical robot.

6 References

1. Y. Nakamura and H. Hanafusa, "Inverse kinematic solutions with singularity robustness for robot manipulator control," *J. Dyn. Sys. Meas. Control*, 108(3), 1986.
2. C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Trans. SMC*, 16(1), 1986.

3. S. Chiaverini, "Singularity-robust task-priority redundancy resolution for real-time kinematic control of robot manipulators," *IEEE Trans. Robotics and Automation*, 13(3), 1997.

4. B. Siciliano and O. Khatib (eds.), *Springer Handbook of Robotics*, 2nd ed., 2016, ch. 8 (Motion Control).

5. E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," *IROS*, 2012.